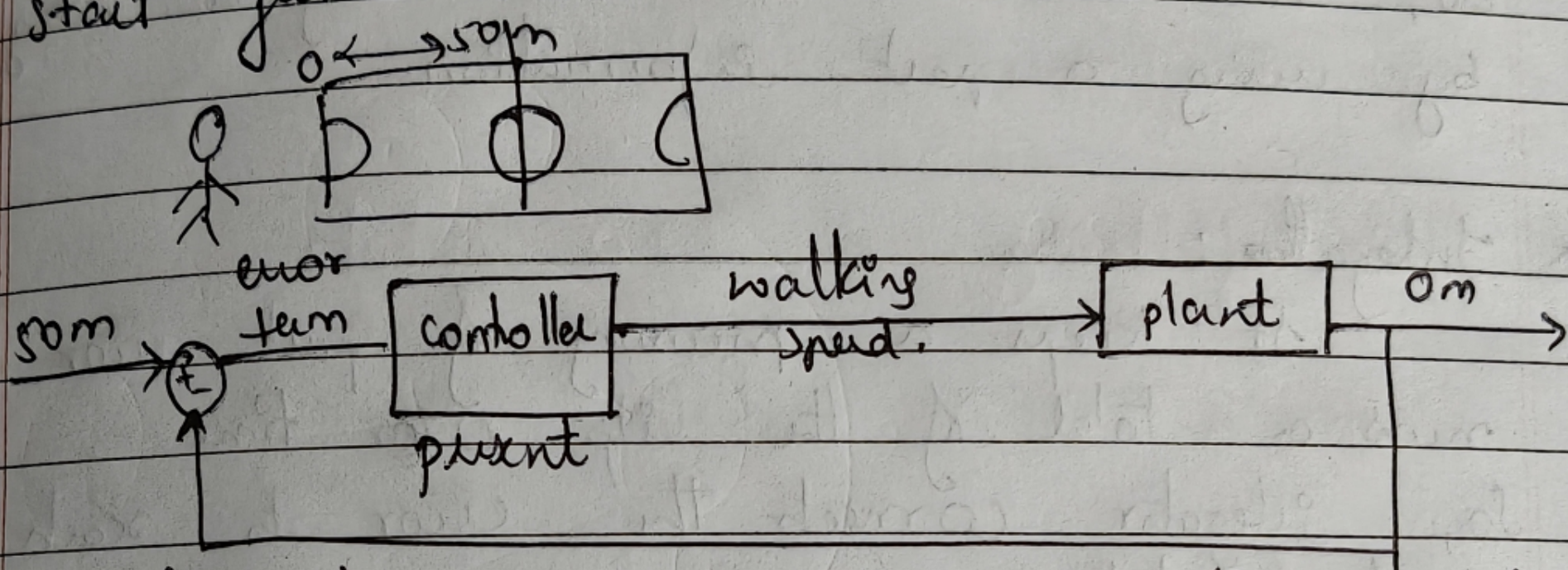


* PID control:

Proportionality (P) gives the present state error factor.

Suppose a plant need to be controlled to get a o/p of 50 m [destination].
So En:-

A boy in football pitch needs to start from 0m to 50m.



So, the boy's brain here is the controller.
So, to reach the 50m, from 0m, error term is 50. So, the brain decides to know what speed he needs to move.

So,

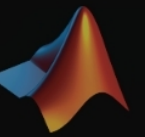
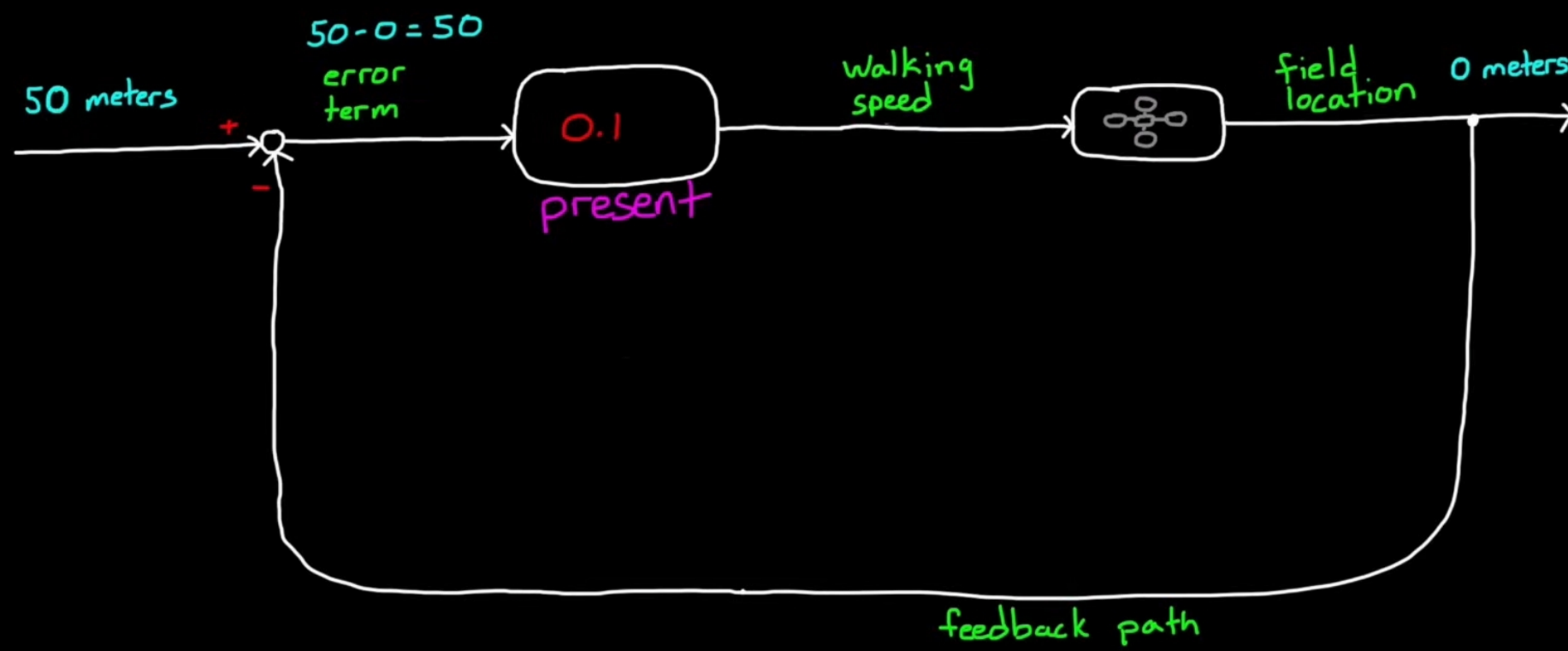
$$(50) \text{ error term} \times 0.1 = 5 \text{ m/s.} \\ (\text{constant})$$

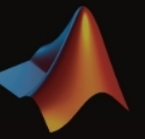
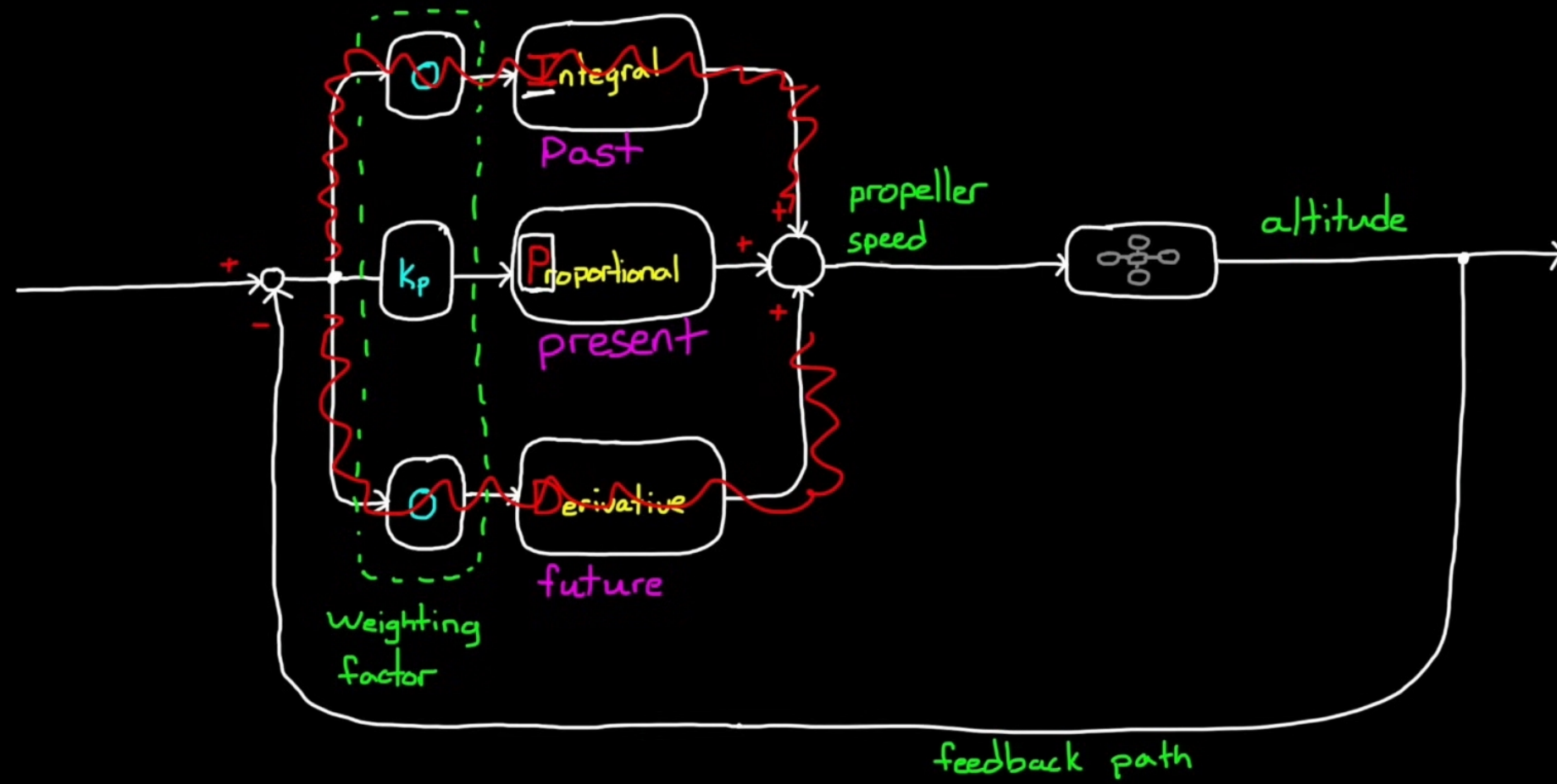
So, the error term tells how much distance is covered.

Similarly in case of drone:

Drone's propeller speed is what analogous to walking speed. So, to move a altitude of 50m, the propellers will spin up, drone will rise, reducing the error.

If it gets all way up to 50m, Error will be 0, propeller will stop and drone will fall. This can will have some error and rise up. So this keeps happening. But the error





Keeps reducing.

Assuming 100 rpm to have

$$\text{Error} \times \text{gain} = \text{speed}$$

$$80 \times 2 = 100 \text{ rpm}$$

$$20 \times 5 = 100 \text{ rpm}$$

$$10 \times 10 = 100 \text{ rpm}$$

$$1 \times 100 = 100 \text{ rpm}$$

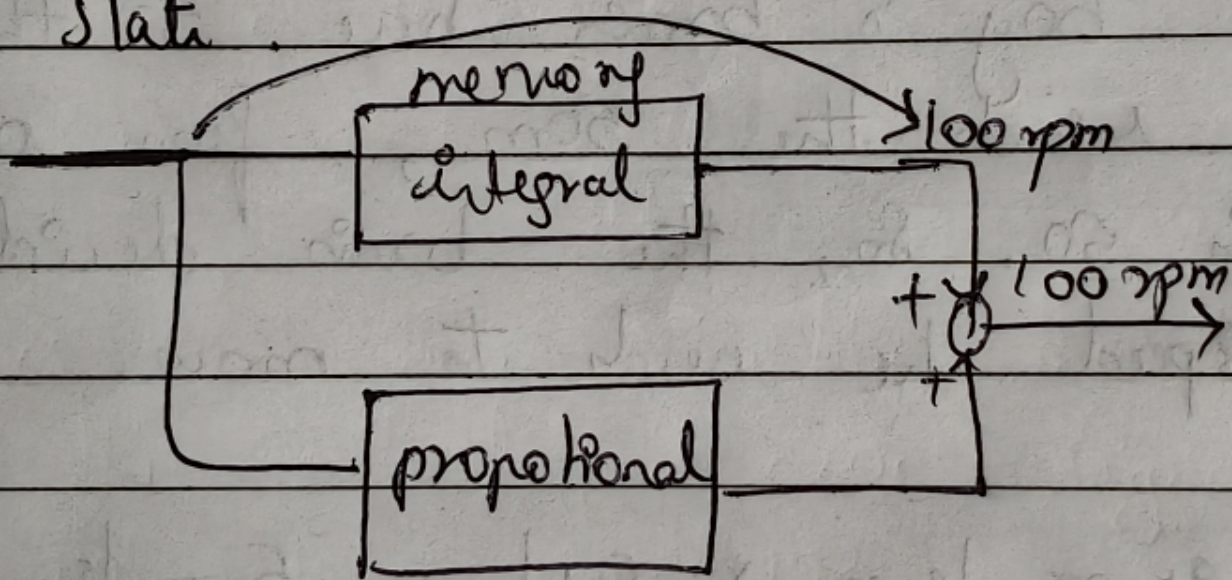
So, gain $\uparrow$, but
error gets $\downarrow$ but
don't go away.

Thus has a steady state error.

So, this can be controlled using a integrator
by using a past information.

* Integrator:

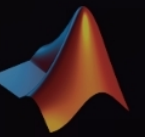
It keeps a memory of past error and
runs a total of the slip over time
So, integrator corrects the error to reach a
steady state.



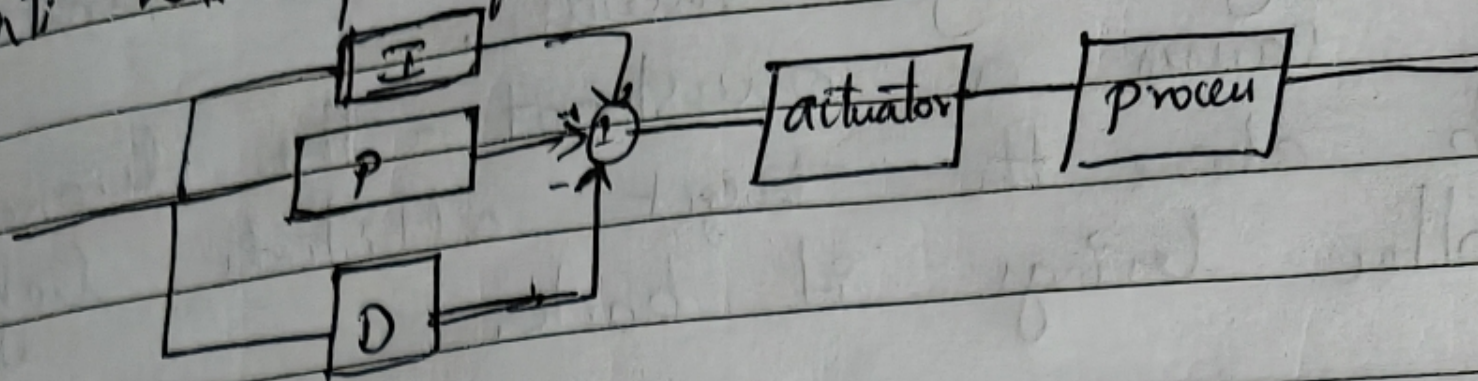
* Derivative:

As the drone goes higher, it overshoots
to maintain a -ve error and stabilizes the
speed. This rise above the desired slip to
maintain a steady state in future is called
a Derivative. This responds to rate of
change of error, seeing how fast the
error is growing or shrinking.

* So, by gain tuning in each branch, we
can control the system error to reach slip



* Anti-Windup for PID control:



In real life, actuators are not linear devices. It takes arbitrary, non-linear paths. It converts the command into response. Thus the actuators in real-life are not linear. It goes to saturation.

Windup occurs when the integral term in a PID controller keeps accumulating error even though the actuator has reached its maximum or minimum limit.

When windup occurs?

In a drone, integral windup occurs the PID controller asks for the motor/ESCs for more control effort than they can physically provide. Suppose the drone is commanded to climb quickly,

⇒ target altitude = 10m

⇒ Actual altitude = 5m

⇒ Error = 5m

The altitude PID increases throttle to make the drone climb.

The motors reach maximum throttle.

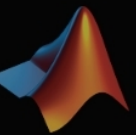
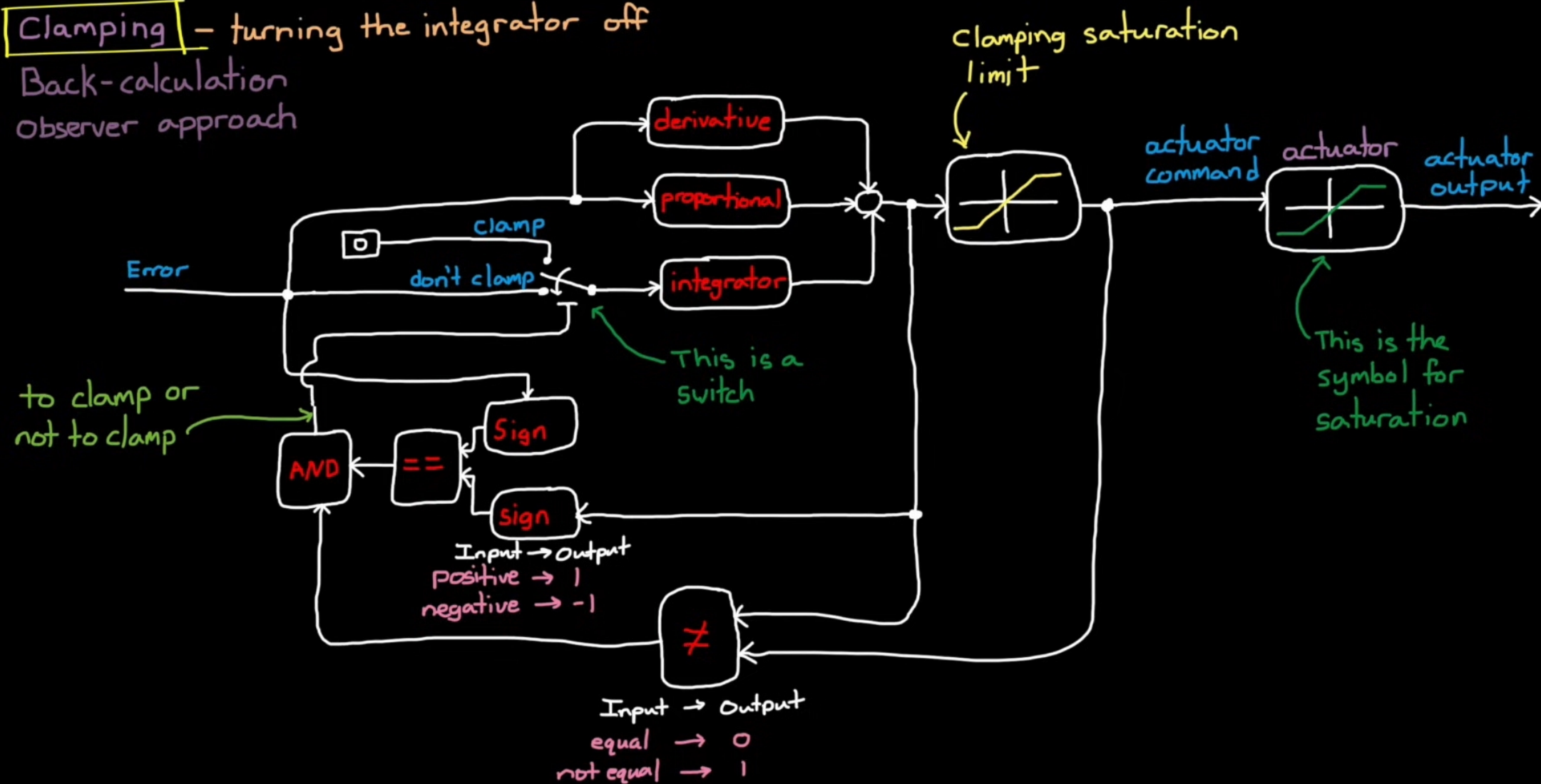
But the drone is still below 10m, so the error remains. The integral term keeps accumulating this error → integral windup.

When the drone finally reaches the desired altitude, the accumulated integral term may

Integrator anti-windup - keep the integrated value from increasing past some specified limit.

Clamping - turning the integrator off

Back-calculation
Observer approach



Page No.
 Date / /
 Skill be large, so the controller continues commanding high throttle.

This causes a overshoot - the drone may climb above the target altitude before the controller brings it back.

⇒ Methods to overcome/avoid:

- * Clamping method.

- * Back-calculation

- * Observed approach.

* Noise filtering in PID control:

In a PID controller, noise filtering is especially important for the derivative (D) term. The derivative term is:

$$D = k_d \frac{de(t)}{dt}$$

It looks at how quickly the error is changing. Suppose the actual sensor signal is smooth.

100 → 101 → 102 → 103

But a sensor has small noise.

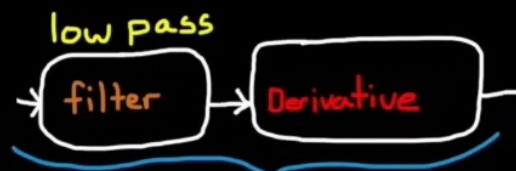
100 → 103 → 101 → 104

The derivative sees these sudden changes as very large rates of changes and produces a large D output.

Small meaning noise → large derivative response.

→ actuator/motor commands fluctuate.

This is particularly important in drones because IMU, accelerometer, gyro, GPS, and barometer measurements contain noise.



let's look at the structure of this

Laplace domain transfer function

S

what it is

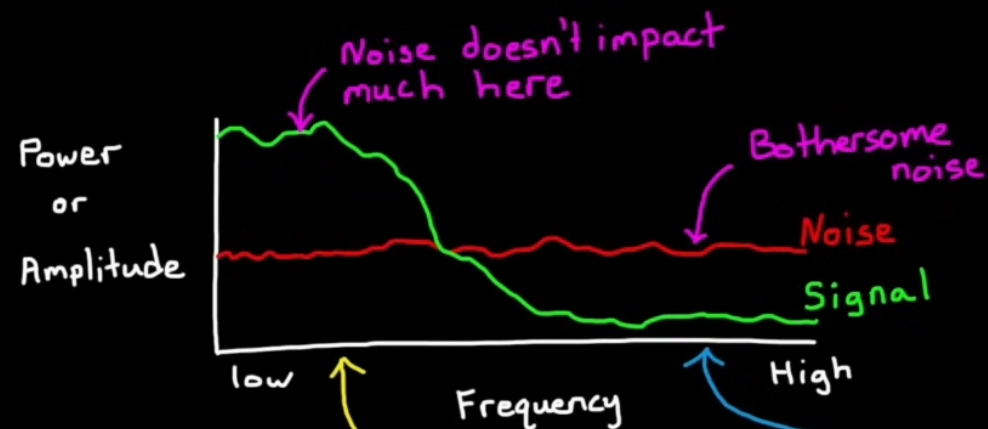
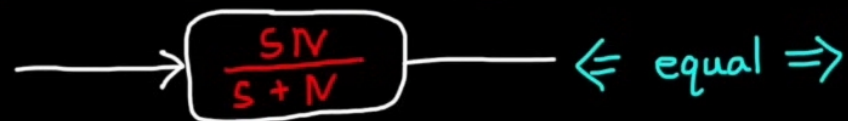
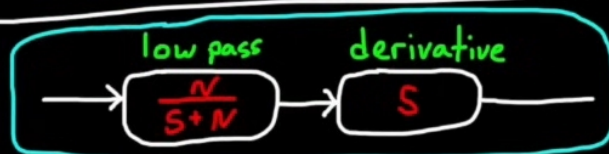
derivative

$\frac{1}{S}$

integral

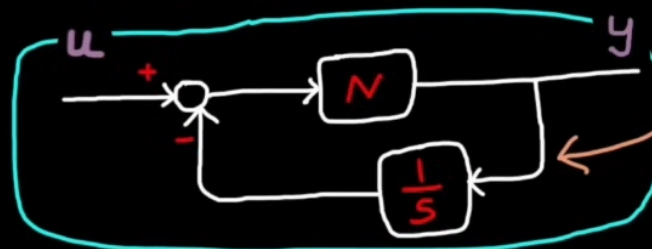
$\frac{N}{S+N}$

low pass filter with cut off at N rad/s



low pass filter - pass low frequencies
attenuate high frequencies

An alternative approach:



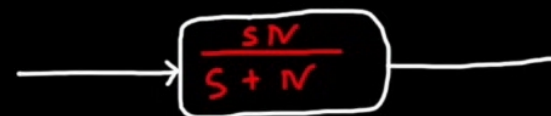
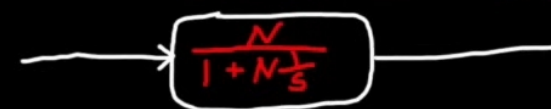
An integral in the feedback path

$$y = N \left(u - \frac{y}{S} \right)$$

$$y + \frac{Ny}{S} = Nu$$

$$y = \frac{N}{1 + N\frac{1}{S}} u$$

$$\boxed{\frac{y}{u} = \frac{N}{1 + N\frac{1}{S}}}$$



Derivative filtering:

$$D(s) = k_d \frac{Ns}{s+N}$$

$N \Rightarrow$ cutoff frequency.

low-frequency changes $\rightarrow$ derivative responds.
High-frequency noise $\rightarrow$ filtered / suppressed.

Thus with a filter with derivative reduces the noise for future error correction.
Two methods can be adopted:

1. By low pass + derivative
2. By feedback path.

PID tuning guide

